

UART Transmitter and Receiver Design Using Verilog HDL

VLSI Internship Project Report

Name: Vasamshetti Meghana

Intern ID: CITS691

Branch: ECE

Duration: 6 Weeks

Abstract

Universal Asynchronous Receiver Transmitter (UART) is a widely used serial communication protocol in digital and embedded systems due to its simplicity and low hardware requirement. It enables asynchronous full-duplex communication between two devices using only two signal lines: transmit (TX) and receive (RX), without requiring a shared clock signal.

This project focuses on the design and implementation of a UART communication system using Verilog Hardware Description Language (HDL). The system is designed according to the standard 8-bit data format with no parity and one stop bit (8N1 configuration). The architecture consists of three main modules: a baud rate generator, a UART transmitter, and a UART receiver.

The baud rate generator provides precise timing control required for serial data transmission. The transmitter converts parallel input data into serial format, while the receiver reconstructs the serial data back into parallel form. The complete system is verified using simulation with loopback testing, where the transmitted signal is directly connected to the receiver input.

The correctness of the design is validated by comparing transmitted and received data, ensuring accurate and error-free communication. This implementation demonstrates an efficient and FPGA-synthesizable UART system suitable for digital communication applications.

1 Introduction

In modern digital communication systems, efficient data transfer between hardware modules is essential. While parallel communication allows high-speed data transfer, it requires multiple data lines, increasing cost, power consumption, and hardware complexity. To overcome these limitations, serial communication protocols are widely used in embedded systems and FPGA-based designs.

One of the most popular serial communication protocols is UART (Universal Asynchronous Receiver Transmitter). UART is a hardware communication protocol that supports asynchronous serial communication, meaning it does not require a shared clock signal between transmitter and receiver. Instead, both devices synchronize data transmission using a pre-defined baud rate.

UART is widely used in microcontrollers, embedded systems, FPGA designs, GPS modules, Bluetooth modules, and computer communication interfaces. It provides a simple and reliable method for data exchange using only two main signal lines: Transmit (TX) and Receive (RX).

A standard UART frame typically consists of:

- 1 Start bit (logic 0)
- 8 Data bits (LSB first transmission)
- Optional Parity bit (not used in this project)
- 1 Stop bit (logic 1)

This project implements a UART system using Verilog HDL, which includes three main functional blocks: baud rate generator, transmitter module, and receiver module. The baud rate generator ensures correct timing for bit transmission, while the transmitter converts 8-bit parallel data into serial form. The receiver performs the reverse operation by sampling incoming serial data and reconstructing the original byte.

The system is verified using simulation tools by applying loopback testing, where the transmitter output is directly connected to the receiver input. The correctness of the system is confirmed when the received data matches the transmitted data.

This design is suitable for FPGA implementation and forms the basis for more advanced communication systems such as RS-232, RS-485, and SoC-level interconnects.

2 Tools Used

The design, simulation, and verification of the UART communication system were carried out using industry-standard digital design tools and hardware description methodologies. The following tools and technologies were used in this project:

- **Verilog HDL (Hardware Description Language):** Verilog HDL was used to design and model the UART transmitter, UART receiver, and baud rate generator at the Register Transfer Level (RTL). It allows the description of digital circuits in terms of logic behavior and timing, making it suitable for FPGA-based implementation.
- **Xilinx Vivado Design Suite:** Vivado is a comprehensive FPGA design tool used for writing, simulating, synthesizing, and verifying Verilog HDL code. In this project, Vivado was used to perform behavioral simulation and functional verification of the UART system.

The tool provides:

- RTL code development environment
 - Built-in Verilog simulator
 - Waveform analysis using simulation results
 - Debugging support for signal-level verification
- **Behavioral Simulation:** Behavioral simulation was performed to verify the functionality of the UART modules before hardware implementation. The simulation results were analyzed using waveform viewers to ensure correct serial transmission and reception of data.

3 Objectives

The main objectives of this project are as follows:

- To design and implement a UART (Universal Asynchronous Receiver Transmitter) communication system using Verilog HDL.
- To develop a baud rate generator for generating accurate timing pulses required for serial data transmission.
- To design a UART transmitter that converts 8-bit parallel data into serial data format.
- To design a UART receiver that converts received serial data back into 8-bit parallel data.
- To verify the functionality of the UART system using simulation and loopback testing.
- To ensure correct data transmission and reception without errors under the 8N1 configuration (8 data bits, No parity, 1 stop bit).

- To analyze waveform outputs and validate the correctness of the design using Vivado simulation tools.

4 UART Overview

Universal Asynchronous Receiver Transmitter (UART) is a widely used serial communication protocol that enables data exchange between digital systems. It is an asynchronous communication method, meaning that no shared clock signal is required between the transmitter and receiver. Instead, both devices agree on a fixed baud rate for synchronization.

UART converts parallel data from a processor or microcontroller into serial form for transmission over a communication line. At the receiver side, the serial data is converted back into parallel form for further processing.

A typical UART communication system consists of two main lines:

- **TX (Transmit line):** Used for sending serial data.
- **RX (Receive line):** Used for receiving serial data.

UART communication follows a standard frame format, which includes:

- 1 Start bit (logic 0)
- 8 Data bits (LSB first transmission)
- No parity bit (in this project)
- 1 Stop bit (logic 1)

This project implements a full UART system using Verilog HDL, including a baud rate generator, transmitter, and receiver modules. The system is verified using loopback simulation, where the transmitted output is directly fed back to the receiver input to validate correct operation.

UART is widely used in embedded systems, FPGA designs, microcontrollers, GPS modules, Bluetooth modules, and computer peripherals due to its simplicity and reliability. “latex

5 UART Architecture

The UART communication system is designed using a modular architecture consisting of three main functional blocks: Baud Rate Generator, UART Transmitter (TX), and UART Receiver (RX). These modules work together to enable reliable asynchronous serial communication between two devices.

The architecture of the UART system is shown in Figure 1.

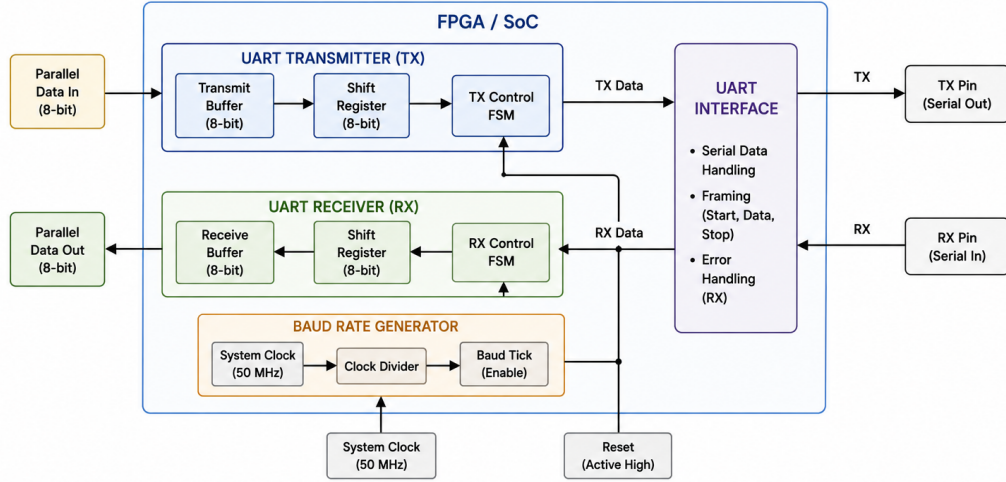


Figure 1: UART Communication System Architecture Block Diagram

5.1 Baud Rate Generator

The baud rate generator produces timing signals required for serial communication. It divides the system clock frequency to generate a baud tick signal, which determines the transmission speed (e.g., 9600 bps). Both transmitter and receiver use this signal for synchronization.

5.2 UART Transmitter (TX)

The transmitter module converts 8-bit parallel data into serial data. It adds a start bit at the beginning and a stop bit at the end of each data frame. The data is transmitted bit by bit in LSB-first order using a finite state machine (FSM) consisting of IDLE, START, DATA, and STOP states.

5.3 UART Receiver (RX)

The receiver module receives serial data from the RX line and converts it back into parallel form. It detects the start bit, samples incoming data using the baud tick, and reconstructs the original 8-bit data. The receiver FSM ensures proper synchronization and error-free data recovery.

5.4 System Operation

The UART system operates in a loopback configuration for verification, where the TX output is directly connected to the RX input. This allows functional testing of the complete communication system without external hardware.

6 RTL Design

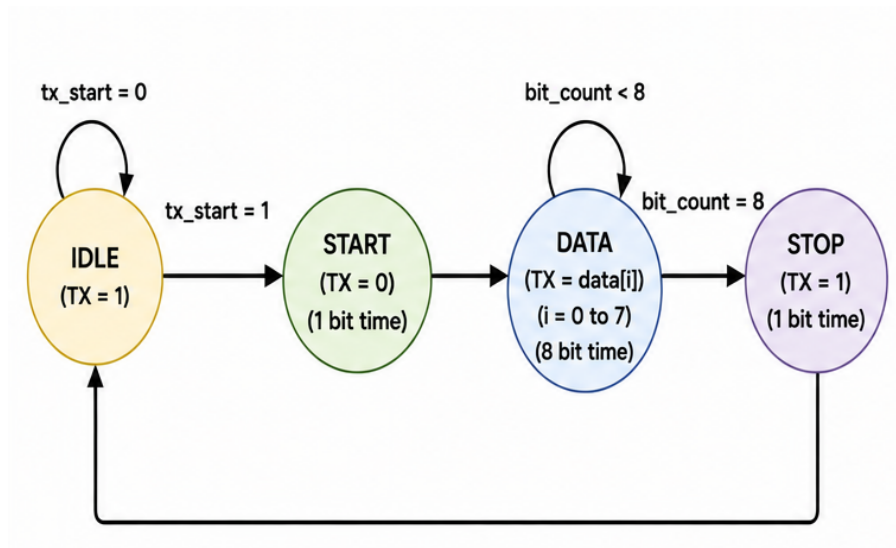
The UART system is implemented at Register Transfer Level (RTL) using Verilog HDL. The design consists of three main modules: Baud Rate Generator, UART Transmitter, and UART Receiver.

6.1 Baud Rate Generator

The baud rate generator is responsible for generating timing pulses required for serial communication. It divides the system clock frequency to produce a baud tick signal, which determines the transmission speed.

6.2 UART Transmitter FSM

The UART transmitter converts 8-bit parallel input data into serial data format.

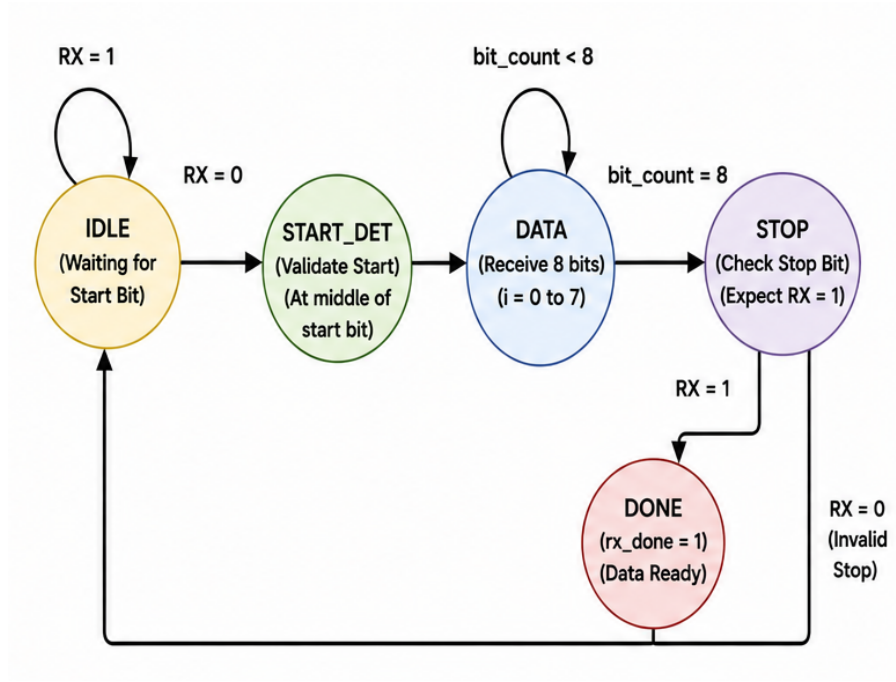


The transmitter FSM consists of the following states:

- **IDLE:** Waits for transmission request.
- **START:** Sends the start bit (logic 0).
- **DATA:** Transmits 8 data bits in LSB-first order.
- **STOP:** Sends the stop bit (logic 1) and returns to IDLE.

6.3 UART Receiver FSM

The receiver converts serial input data back into 8-bit parallel form.



The receiver FSM consists of the following states:

- **IDLE:** Waits for start bit detection.
- **START:** Confirms the start bit.
- **DATA:** Receives 8 serial bits.
- **STOP:** Verifies the stop bit.
- **DONE:** Stores received data and asserts the receive-done signal.

7 Verilog HDL Implementation

The UART communication system was implemented using Verilog Hardware Description Language (HDL) at the Register Transfer Level (RTL). The design follows a modular approach, where each functional block is developed as an independent module and later integrated into a complete UART system.

The UART design supports the standard 8N1 communication format, which consists of one start bit, eight data bits, no parity bit, and one stop bit. The system operates asynchronously, meaning that no external clock signal is shared between the transmitter and receiver. Instead, both modules are synchronized using a predefined baud rate generated internally by the baud rate generator.

The complete UART system consists of the following modules:

- **Baud Rate Generator**

- **UART Transmitter (TX)**
- **UART Receiver (RX)**
- **Top Module**

Each module was designed, simulated, and verified individually before integrating them into the complete UART communication system.

The baud rate generator produces timing pulses by dividing the system clock frequency to generate the required baud tick signal. This timing signal controls the transmission and reception processes.

The UART transmitter converts 8-bit parallel input data into serial format by adding the start bit and stop bit according to the UART protocol. The transmitter operation is controlled using a finite state machine (FSM) consisting of IDLE, START, DATA, and STOP states.

The UART receiver detects the start bit, samples the incoming serial data at appropriate time intervals, reconstructs the original 8-bit data, and generates a receive-complete signal after successful reception. The receiver operation is also controlled using a finite state machine.

The top module integrates the baud rate generator, transmitter, and receiver modules into a complete UART communication system. For functional verification, a loopback configuration is implemented by connecting the transmitter output directly to the receiver input.

The complete UART design was verified using a Verilog testbench in Xilinx Vivado Design Suite. Simulation results confirmed successful transmission and reception of data with accurate timing and error-free communication.

The following subsections present the Verilog code implementation of each module.

7.1 Baud Rate Generator

```
module baud_generator (
    parameter CLK_FREQ = 50000000,
    parameter BAUD_RATE = 9600
) (
    input wire clk,
    input wire rst,
    output reg baud_tick
);

localparam DIVISOR = CLK_FREQ / BAUD_RATE;

reg [15:0] count;
```



```

always @(posedge clk or posedge rst) begin
    if (rst) begin
        count <= 0;
        baud_tick <= 0;
    end
    else if (count == DIVISOR-1) begin
        count <= 0;
        baud_tick <= 1;
    end
    else begin
        count <= count + 1;
        baud_tick <= 0;
    end
end
endmodule

```

The baud rate generator divides the system clock frequency to generate a baud tick signal required for UART communication.

7.2 UART Transmitter

```

module uart_tx(
    input wire      clk,
    input wire      rst,
    input wire      baud_tick,
    input wire      tx_start,
    input wire [7:0] data_in,

    output reg      tx,
    output reg      tx_busy
);

localparam IDLE  = 2'd0,
            START = 2'd1,
            DATA = 2'd2,
            STOP  = 2'd3;

reg [1:0] state;
reg [7:0] shift_reg;
reg [2:0] bit_count;

```

```

always @(posedge clk or posedge rst) begin
    if (rst) begin
        state      <= IDLE;
        tx         <= 1'b1;
        tx_busy    <= 1'b0;
        shift_reg  <= 8'd0;
        bit_count  <= 3'd0;
    end
    else begin
        case (state)

            IDLE: begin
                tx <= 1'b1;
                tx_busy <= 1'b0;

                // Check tx_start every clock cycle
                if (tx_start) begin
                    shift_reg <= data_in;
                    bit_count <= 3'd0;
                    tx_busy    <= 1'b1;
                    state      <= START;
                end
            end

            START: begin
                if (baud_tick) begin
                    tx <= 1'b0;
                    state <= DATA;
                end
            end

            DATA: begin
                if (baud_tick) begin
                    tx <= shift_reg[0];
                    shift_reg <= shift_reg >> 1;

                    if (bit_count == 3'd7)
                        state <= STOP;
                    else
                        bit_count <= bit_count + 1;
                end
            end
        end
    end
end

```

```

        STOP: begin
            if (baud_tick) begin
                tx <= 1'b1;
                tx_busy <= 1'b0;
                state <= IDLE;
            end
        end
    end

endcase
end
end

endmodule

```

The UART transmitter converts 8-bit parallel data into serial format by adding start and stop bits.

7.3 UART Receiver

```

module uart_rx(
    input wire      clk,
    input wire      rst,
    input wire      baud_tick,
    input wire      rx,

    output reg [7:0] data_out,
    output reg      rx_done
);

localparam IDLE = 2'd0,
            DATA = 2'd1,
            STOP = 2'd2,
            DONE = 2'd3;

reg [1:0] state;
reg [2:0] bit_count;
reg [7:0] shift_reg;

always @(posedge clk or posedge rst) begin
    if (rst) begin
        state <= IDLE;
        bit_count <= 3'd0;
        shift_reg <= 8'd0;
    end
end

```

```

        data_out  <= 8'd0;
        rx_done   <= 1'b0;
    end
    else begin
        rx_done <= 1'b0;

        if (baud_tick) begin
            case (state)

                IDLE: begin
                    if (rx == 1'b0) begin
                        bit_count <= 3'd0;
                        state <= DATA;
                    end
                end

                DATA: begin
                    shift_reg[bit_count] <= rx;

                    if (bit_count == 3'd7)
                        state <= STOP;
                    else
                        bit_count <= bit_count + 1;
                    end

                STOP: begin
                    if (rx == 1'b1)
                        state <= DONE;
                    else
                        state <= IDLE;
                    end

                DONE: begin
                    data_out <= shift_reg;
                    rx_done <= 1'b1;
                    state <= IDLE;
                end

            endcase
        end
    end
end
end

```

```
endmodule
```

The UART receiver samples the incoming serial data and reconstructs the original 8-bit parallel data.

7.4 Top Module

```
module uart_top(  
    input  wire      clk,  
    input  wire      rst,  
    input  wire      tx_start,  
    input  wire [7:0] data_in,  
    input  wire      rx,  
  
    output wire      tx,  
    output wire [7:0] data_out,  
    output wire      rx_done  
);  
  
wire baud_tick;  
  
baud_generator baud_gen (  
    .clk(clk),  
    .rst(rst),  
    .baud_tick(baud_tick)  
);  
  
uart_tx tx_inst (  
    .clk(clk),  
    .rst(rst),  
    .baud_tick(baud_tick),  
    .tx_start(tx_start),  
    .data_in(data_in),  
    .tx(tx),  
    .tx_busy()  
);  
  
uart_rx rx_inst (  
    .clk(clk),  
    .rst(rst),  
    .baud_tick(baud_tick),  
    .rx(rx),  
    .data_out(data_out),
```

```

        .rx_done(rx_done)
    );

endmodule

```

The top module integrates the baud rate generator, transmitter, and receiver modules into a complete UART communication system.

8 Testbench Design

A Verilog testbench was developed to verify the functionality of the complete UART system. Loopback testing was performed by connecting the transmitter output directly to the receiver input.

```

`timescale 1ns/1ps

module uart_tb;

    reg        clk;
    reg        rst;
    reg        tx_start;
    reg [7:0] data_in;

    wire        tx;
    wire        rx;
    wire [7:0] data_out;
    wire        rx_done;

    // Loopback connection: TX -> RX
    assign rx = tx;

    // DUT instantiation
    uart_top dut (
        .clk(clk),
        .rst(rst),
        .tx_start(tx_start),
        .data_in(data_in),
        .rx(rx),
        .tx(tx),
        .data_out(data_out),
        .rx_done(rx_done)
    );

```

```

// 50 MHz clock generation (20 ns period)
initial
    clk = 1'b0;

always #10 clk = ~clk;

initial begin

    // Initialize signals
    rst      = 1'b1;
    tx_start = 1'b0;
    data_in  = 8'h00;

    // Hold reset
    #100;
    rst = 1'b0;

    // Wait for stabilization
    #100;

    // Transmit one byte
    data_in  = 8'h55;
    tx_start = 1'b1;

    // Keep tx_start high long enough
    #200000;

    tx_start = 1'b0;

    // Wait for reception
    wait (rx_done == 1'b1);

    $display("Time = %0t ns, Received Data = %h",
             $time, data_out);

    #1000;
    $finish;
end

endmodule

```

The testbench applies different input data patterns and verifies that the received data matches the transmitted data.

9 Simulation Results

The UART design was simulated and verified using the Xilinx Vivado Design Suite. Behavioral simulation was performed to validate the functionality of the baud rate generator, UART transmitter, UART receiver, and the integrated top module. A loopback testing methodology was adopted, where the transmitter output (TX) was directly connected to the receiver input (RX). This configuration enables complete verification of the UART communication system without requiring any external hardware. Various 8-bit data patterns were applied through the testbench to evaluate the transmission and reception processes under the standard 8N1 UART configuration, which consists of one start bit, eight data bits, no parity bit, and one stop bit. The simulation results confirmed that:

- The baud rate generator produced accurate timing pulses.
- The transmitter correctly serialized the input data.
- The receiver successfully detected the start bit and sampled incoming data at the correct intervals.
- The received data matched the transmitted data for all test cases.
- The UART system achieved reliable and error-free communication.

9.1 UART Transmitter Waveform

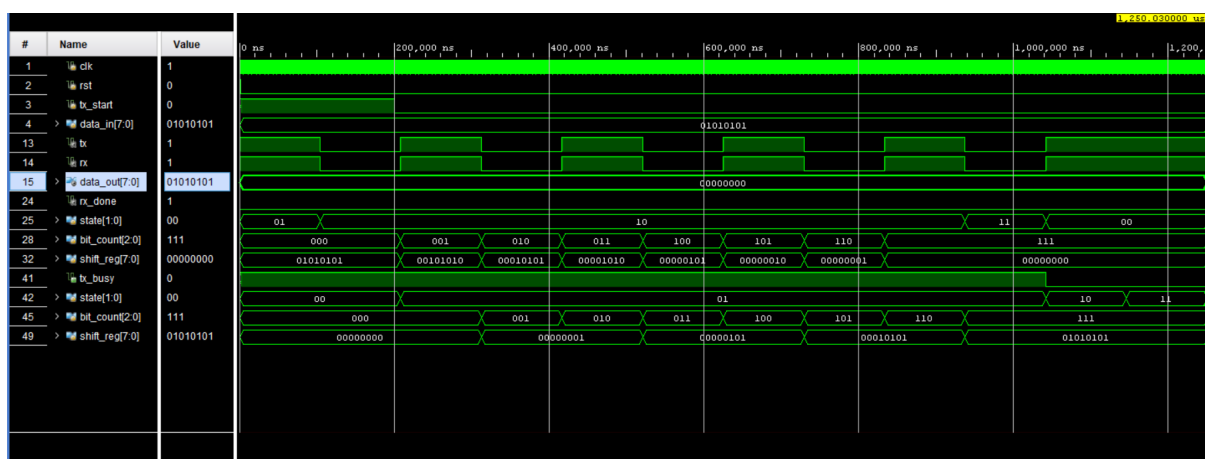


Figure 2: UART Transmitter Simulation Waveform

9.2 UART Receiver Waveform

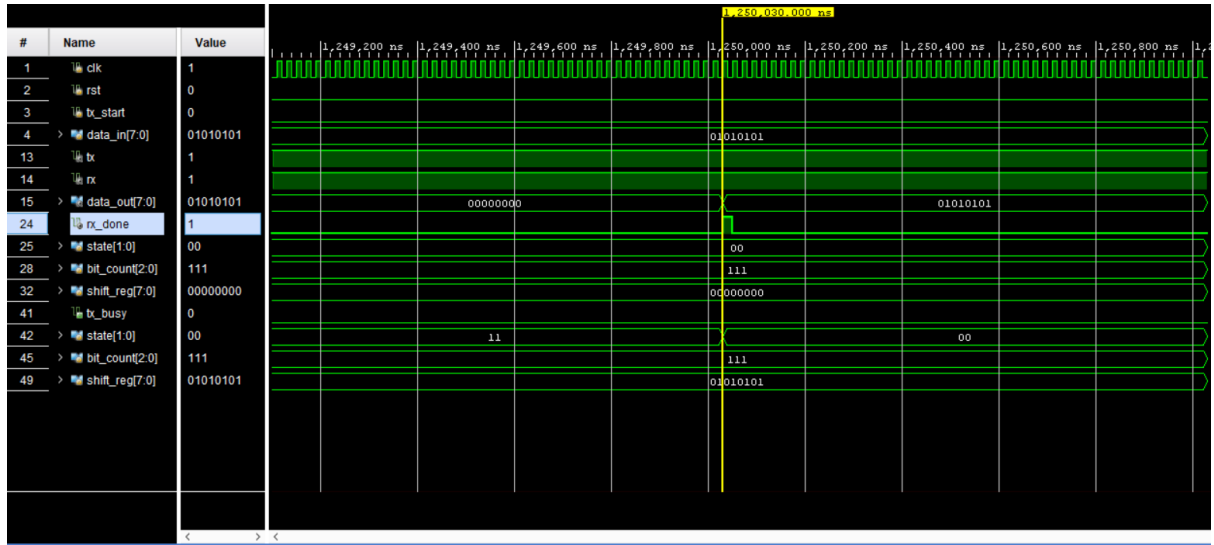
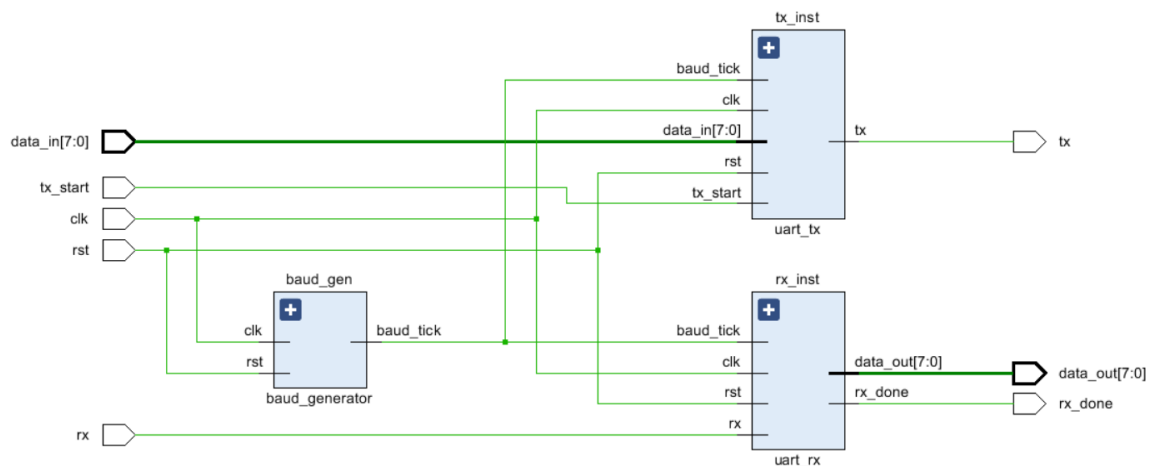


Figure 3: UART Receiver Simulation Waveform

The simulation results confirm correct transmission and reception of serial data under the 8N1 configuration. “ latex

10 RTL Schematic

The RTL schematic represents the synthesized hardware structure of the UART system. It shows the internal interconnection between the baud rate generator, UART transmitter, UART receiver, registers, counters, and control logic.



11 Synthesis Results

The UART design was synthesized using Xilinx Vivado Design Suite to analyze hardware resource utilization and timing performance.

The synthesis report indicates that the design requires minimal FPGA resources due to its simple FSM-based architecture and shift-register implementation.

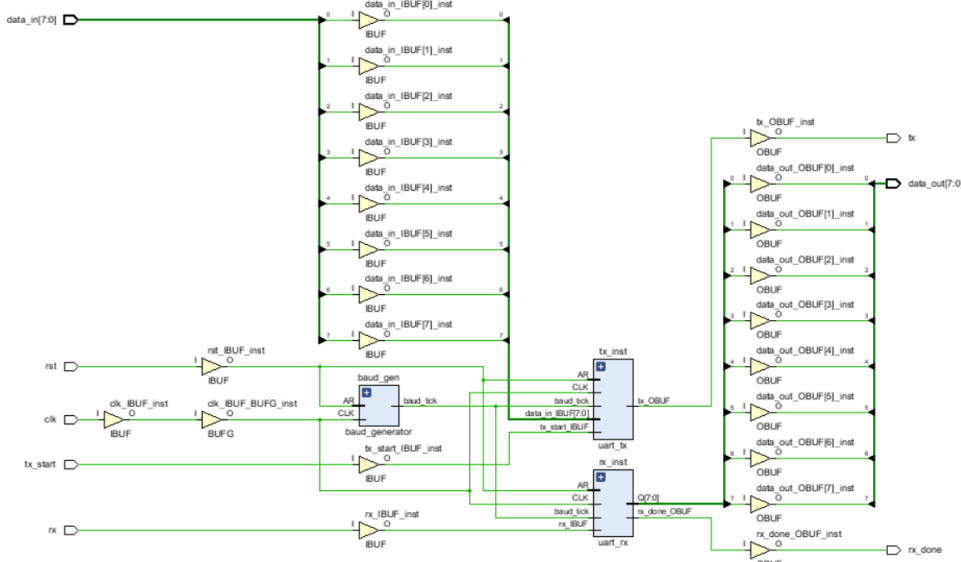


Figure 4: UART Synthesis Report Showing Resource Utilization

Parameter	Description
Flip-Flops (FFs)	Used for storing state information and serial data.
Slices	Represents the overall FPGA resource utilization.
Power Consumption	Overall power consumed by the UART system.

Table 1: UART Synthesis Summary

12 Results

The UART communication system was successfully designed and implemented using Verilog HDL.

Functional verification was performed through loopback testing, where the transmitter output was directly connected to the receiver input.

The simulation results confirmed that:

- The transmitter correctly converted 8-bit parallel data into serial format.
- The receiver accurately reconstructed the original data.

- The baud rate generator provided precise timing signals.
- The transmitted and received data matched successfully.
- The UART system operated correctly using the 8N1 protocol configuration.

13 Conclusion

The UART transmitter and receiver system was successfully designed and verified using Verilog HDL and Xilinx Vivado Design Suite.

The baud rate generator ensured accurate timing, while the transmitter and receiver modules enabled reliable asynchronous communication.

Simulation and loopback testing confirmed error-free transmission and reception of serial data. The design is FPGA synthesizable and suitable for embedded communication applications.

14 Advantages

- Simple and efficient communication protocol.
- Requires only TX and RX lines.
- No external clock signal is required.
- Low hardware complexity and resource utilization.
- Easy integration with microcontrollers and FPGAs.
- Suitable for low-power embedded systems.

15 Applications

- Microcontroller communication.
- FPGA-based embedded systems.
- GPS modules.
- Bluetooth and Wi-Fi modules.
- Serial debugging interfaces.
- Industrial automation systems.

16 Future Scope

- Support for configurable baud rates.
- Addition of parity-bit error detection.
- FPGA hardware implementation and validation.
- Integration into System-on-Chip (SoC) platforms.
- Support for advanced protocols such as RS-232 and RS-485.
- Development of multi-UART communication systems.

17 References

1. Samir Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, Pearson Education.
2. M. Morris Mano and Michael D. Ciletti, *Digital Design*, Pearson.
3. Xilinx Vivado Design Suite User Guide.
4. UART Communication Protocol Specification.
5. FPGA-Based System Design Documentation.